

# TalentAI

## API Specification

*Companion to the TalentAI BRD/FRD, Business Rules & Acceptance Criteria, NFR Addendum, and Implementation Plan. Base URL: <https://api.talentai.com/v1>*

Version 1.0

Prepared by: Business Analysis Team

Date: August 29, 2026

Status: Draft — For Solution Architect Review

## Changes from the Original Draft

Merged: `/ai/resume-parser` into `POST /candidates/{candidateId}/resume/parse` — duplicate endpoint for the same action.

Consolidated: Shortlist / Reject / Approve (previously spread across "Recruiter APIs" and "Hiring Manager APIs") into one canonical `PATCH /applications/{applicationId}/status` endpoint — these were the same pipeline state-transition operation performed at different stages by different roles, not separate APIs.

Merged: Send Email + Send SMS into one `POST /notifications` with a channel field.

Changed: `DELETE /users/{userId}` → `PATCH /users/{userId}/status` — users are deactivated, not hard-deleted, to preserve the audit trail (BR-006).

Changed: PUT/DELETE for interview reschedule/cancel → `PATCH .../reschedule` and `.../cancel` — these are state changes with a reason, not a record replacement or deletion.

Added: `POST /users` for administrator-created internal accounts, distinct from candidate self-registration (BR-003).

Added: Requisition approval workflow (submit / approve) — without it, Publish Job had no BR-008 enforcement point.

Added: Screening Questions domain — Phase 5 of the BRD (FR-10, FR-11, US-031–034) had no API representation.

Added: Background Verification and Onboarding & Pre-boarding domains — both are Must-Have FR modules and existing business domains in this document set, but were absent from the original catalog.

Added: full response body for AI Candidate Matching, including the matched/partial/missing breakdown required by Constraint C04 — the original draft left this endpoint's response undefined.

Removed: the "Recruiter APIs" and "Hiring Manager APIs" sections as standalone groupings — their actions now live under Application/Offer Management with role-based authorization, per BR-002.

Removed: the BA-vs-architect lifecycle worked examples from the companion discussion — condensed into a short ownership note in the Appendix, since the worked examples duplicated content already fully specified in this document.

# 1. Authentication APIs

## **POST /auth/register — Register Candidate**

Self-service registration for candidates. Internal roles (Recruiter, Hiring Manager, Interviewer, HR Admin) are created by an administrator via POST /users, not self-registration (BR-003).

### *Request*

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@gmail.com",
  "password": "Password@123",
  "role": "Candidate"
}
```

### *Response*

```
{
  "userId": 101,
  "message": "Registration successful"
}
```

## **POST /auth/login — Login**

### *Request*

```
{
  "email": "john@gmail.com",
  "password": "Password@123"
}
```

### *Response*

```
{
  "token": "JWT_TOKEN",
  "role": "Candidate",
  "expiresIn": 3600
}
```

## **POST /auth/forgot-password — Forgot Password**

### *Request*

```
{ "email": "john@gmail.com" }
```

### *Response*

```
{ "message": "Password reset link sent to registered email." }
```

## **POST /auth/reset-password — Reset Password**

### *Request*

```
{
  "resetToken": "8f2a1c...",
  "newPassword": "NewPass@123"
}
```

### *Response*

```
{ "message": "Password reset successful." }
```

## **POST /auth/logout — Logout**

Requires a valid Authorization header; no request body.

### *Response*

```
{ "message": "Logged out successfully." }
```

## 2. User & Access Management

### **POST** /users — Create Internal User — added

Administrator creates a Recruiter, Hiring Manager, Interviewer, or HR Admin account. Ref: BR-003, US-002.

#### *Request*

```
{
  "firstName": "Priya",
  "lastName": "Nair",
  "email": "priya.nair@company.com",
  "role": "Recruiter",
  "department": "Talent Acquisition"
}
```

#### *Response*

```
{
  "userId": 305,
  "status": "Active",
  "message": "User created successfully."
}
```

### **GET** /users/{userId} — Get User Profile

#### *Response*

```
{
  "userId": 305,
  "firstName": "Priya",
  "lastName": "Nair",
  "email": "priya.nair@company.com",
  "role": "Recruiter",
  "status": "Active"
}
```

### **PUT** /users/{userId} — Update User

#### *Request*

```
{
  "firstName": "Priya",
  "lastName": "Nair",
  "department": "Talent Acquisition"
}
```

#### *Response*

```
{ "userId": 305, "message": "User updated successfully." }
```

### **PATCH** /users/{userId}/status — Activate / Deactivate User — changed from DELETE

Users are deactivated, not hard-deleted, so the audit trail stays intact. Ref: BR-003, BR-004, BR-006.

#### *Request*

```
{ "status": "Deactivated" }
```

#### *Response*

```
{ "userId": 305, "status": "Deactivated" }
```

**GET** /users?role=Recruiter&status=Active&page=1&size=20 — Get All Users

*Response*

```
{
  "totalRecords": 42,
  "page": 1,
  "size": 20,
  "data": [
    { "userId": 305, "firstName": "Priya", "role": "Recruiter", "status": "Active" }
  ]
}
```

### 3. Candidate Management

**POST** /candidates — Create Candidate Profile

*Request*

```
{
  "userId": 101,
  "phone": "+91-9876543210",
  "location": "Bengaluru"
}
```

*Response*

```
{ "candidateId": 201, "message": "Candidate profile created." }
```

**GET** /candidates/{candidateId} — Get Candidate Profile

*Response*

```
{
  "candidateId": 201,
  "name": "John Doe",
  "email": "john@gmail.com",
  "location": "Bengaluru",
  "skills": ["Java", "SQL"],
  "experience": 5,
  "education": "B.Tech",
  "resumeUrl": "/files/resumes/201.pdf"
}
```

**PUT** /candidates/{candidateId} — Update Candidate Profile

*Request*

```
{
  "location": "Hyderabad",
  "noticePeriod": "30 days",
  "salaryExpectation": 1800000
}
```

*Response*

```
{ "candidateId": 201, "message": "Profile updated successfully." }
```

**POST** /candidates/{candidateId}/resume — Upload Resume

multipart/form-data, field name "resume". Supported formats: PDF, DOC, DOCX.

#### Response

```
{ "resumeId": 5001, "fileName": "john_doe_resume.pdf", "status": "Uploaded" }
```

### **POST /candidates/{candidateId}/resume/parse — Parse Resume (AI) — canonical, merged with /ai/resume-parser**

The standalone /ai/resume-parser endpoint from the original draft is removed; this candidate-scoped endpoint is the single entry point for AI resume parsing.

#### Response

```
{
  "candidateId": 201,
  "extracted": {
    "skills": ["Java", "Spring Boot", "SQL"],
    "experience": 5,
    "education": "B.Tech",
    "certifications": ["AWS Certified Developer"],
    "companies": ["Infosys", "TCS"]
  },
  "status": "PendingReview"
}
```

*Extracted data requires recruiter review before overwriting verified profile fields (BR-021, BR-022). If parsing fails, the original resume and candidate record remain available (BR-024).*

### **GET /candidates/{candidateId}/skills — Get Candidate Skills**

#### Response

```
{ "candidateId": 201, "skills": ["Java", "Spring Boot", "SQL", "AWS (basic)"] }
```

### **POST /candidates/{candidateId}/certifications — Add Certification**

#### Request

```
{
  "name": "AWS Certified Developer",
  "issuer": "AWS",
  "issueDate": "2024-03-15"
}
```

#### Response

```
{ "certificationId": 9001, "message": "Certification added." }
```

## 4. Job / Requisition Management

### POST /jobs — Create Job Requisition

#### Request

```
{
  "title": "Senior Java Developer",
  "department": "Engineering",
  "location": "Bengaluru",
  "employmentType": "Full-Time",
  "requiredSkills": ["Java", "Spring Boot", "REST APIs"],
  "preferredSkills": ["AWS", "Kafka"],
  "minExperience": 5,
  "maxExperience": 8,
  "hiringManagerId": 410,
  "recruiterId": 305
}
```

#### Response

```
{ "jobId": 5001, "status": "Draft", "message": "Requisition created." }
```

### GET /jobs/{jobId} — Get Job

#### Response

```
{
  "jobId": 5001,
  "title": "Senior Java Developer",
  "status": "Draft",
  "requiredSkills": ["Java", "Spring Boot", "REST APIs"],
  "preferredSkills": ["AWS", "Kafka"]
}
```

### PUT /jobs/{jobId} — Update Job

Same shape as Create; permitted while the requisition is in Draft.

#### Response

```
{ "jobId": 5001, "message": "Requisition updated." }
```

### PATCH /jobs/{jobId}/submit — Submit for Approval — added

Missing from the original draft — without it, Publish had no approval gate. Ref: BR-007, FR-REQ-01, US-009.

#### Response

```
{ "jobId": 5001, "status": "PendingApproval" }
```

### PATCH /jobs/{jobId}/approve — Approve / Reject Requisition — added

Ref: BR-008, BR-009, FR-REQ-02, US-011.

#### Request

```
{
  "decision": "Approved",
  "comments": "Looks good, proceed to publish."
}
```

#### Response

```
{ "jobId": 5001, "status": "Approved" }
```

#### **PATCH** /jobs/{jobId}/publish — Publish Job

##### *Response*

```
{ "jobId": 5001, "status": "Published" }
```

##### *Error Response*

```
{ "errorCode": "JOB-001", "message": "Requisition must be approved before publishing." }
```

#### **PATCH** /jobs/{jobId}/close — Close Job

##### *Response*

```
{ "jobId": 5001, "status": "Closed" }
```

#### **PATCH** /jobs/{jobId}/archive — Archive Job

##### *Response*

```
{ "jobId": 5001, "status": "Archived" }
```

#### **GET** /jobs?location=Bengaluru&skills=Java&experience=5&status=Published&page=1&size=10 — Search Jobs

##### *Response*

```
{
  "totalRecords": 128,
  "page": 1,
  "size": 10,
  "data": [ { "jobId": 5001, "title": "Senior Java Developer", "location": "Bengaluru" } ]
}
```

## 5. Job Applications & Pipeline

#### **POST** /jobs/{jobId}/apply — Apply for Job

##### *Request*

```
{ "candidateId": 201 }
```

##### *Response*

```
{
  "applicationId": 9001,
  "status": "Applied",
  "message": "Application submitted successfully."
}
```

##### *Error Response*

```
{ "errorCode": "APP-001", "message": "Candidate has already applied for this job." }
```

#### **PATCH** /applications/{applicationId}/withdraw — Withdraw Application

##### *Response*

```
{ "applicationId": 9001, "status": "Withdrawn" }
```

#### **GET** /candidates/{candidateId}/applications — Get Candidate Applications

##### *Response*

```
{
  "data": [
```

```
{ "applicationId": 9001, "jobTitle": "Senior Java Developer", "status": "Shortlisted",  
  "appliedOn": "2026-09-01" }  
]
```

#### **GET** /jobs/{jobId}/applications?status=Shortlisted&page=1&size=20 — Get Applicants

##### **Response**

```
{  
  "totalRecords": 60,  
  "data": [ { "applicationId": 9001, "candidateName": "John Doe", "matchScore": 87, "status":  
    "Shortlisted" } ]  
}
```

#### **PATCH** /applications/{applicationId}/status — Update Application Status — canonical, consolidates Shortlist / Reject / Approve

Single endpoint for every pipeline-stage transition (shortlist, hold, reject, select), authorized by role per BR-002.  
Ref: BR-036, BR-037, BR-038, BR-044, FR-PIPE-02.

##### **Request**

```
{  
  "status": "Rejected",  
  "reasonCode": "SKILLS_MISMATCH",  
  "comments": "Missing required Kafka experience."  
}
```

##### **Response**

```
{ "applicationId": 9001, "status": "Rejected" }
```

*status: Shortlisted | OnHold | Rejected | Selected. reasonCode is mandatory when status = Rejected (BR-037). Replaces the separate /shortlist, /reject, and /approve endpoints from "Recruiter APIs" and "Hiring Manager APIs" in the original draft — those were the same state transition performed at different pipeline stages.*

## 6. Candidate Screening (Screening Questions)

### **POST** /jobs/{jobId}/screening-questions — Configure Screening Questions — added

This entire domain (Phase 5 of the BRD — FR-10, FR-11, US-031–034) had no API representation in the original draft.

#### *Request*

```
{
  "questions": [
    { "text": "Are you willing to relocate to Bengaluru?", "type": "YesNo", "mandatory": true,
      "disqualifyingAnswer": "No" },
    { "text": "Years of Java experience", "type": "Numeric", "mandatory": true, "minValue": 5
    }
  ]
}
```

#### *Response*

```
{ "jobId": 5001, "questionCount": 2, "message": "Screening questions configured." }
```

### **GET** /jobs/{jobId}/screening-questions — Get Screening Questions — added

#### *Response*

```
{
  "jobId": 5001,
  "questions": [ { "questionId": 701, "text": "Are you willing to relocate to Bengaluru?",
    "type": "YesNo", "mandatory": true } ]
}
```

### **POST** /applications/{applicationId}/screening-responses — Submit Screening Responses — added

#### *Request*

```
{
  "responses": [
    { "questionId": 701, "answer": "Yes" },
    { "questionId": 702, "answer": "6" }
  ]
}
```

#### *Response*

```
{ "applicationId": 9001, "status": "ResponsesRecorded" }
```

### **GET** /applications/{applicationId}/screening-results — Get Screening Results — added

#### *Response*

```
{
  "applicationId": 9001,
  "result": "Eligible",
  "mandatoryCriteriaMet": true,
  "details": [ { "questionId": 701, "answer": "Yes", "status": "Pass" } ]
}
```

*BR-035: candidate must satisfy all mandatory eligibility criteria before being classified eligible for shortlisting.*

## 7. AI Candidate Matching & Screening

### **POST** /ai/candidate-match — AI Candidate Matching

#### ***Request***

```
{ "candidateId": 25, "jobId": 100 }
```

#### ***Response***

```
{
  "matchId": 30045,
  "candidateId": 201,
  "jobId": 5001,
  "overallMatch": 87,
  "breakdown": { "skills": 92, "experience": 90, "education": 100, "preferredSkills": 70 },
  "matchedSkills": ["Java", "Spring Boot", "REST APIs", "Microservices"],
  "partialMatches": [ { "skill": "AWS", "note": "Basic experience only" } ],
  "missingSkills": ["Kafka"]
}
```

*The original draft did not define a response for this endpoint. The breakdown/matched/partial/missing structure is mandatory, not optional — Constraint C04 and NFR-AI-02 require every score to carry its supporting factors; a bare number is never returned alone.*

#### **GET /jobs/{jobId}/ranking?sortBy=matchScore&page=1&size=20 — Candidate Ranking**

#### ***Response***

```
{
  "data": [ { "candidateId": 201, "name": "John Doe", "matchScore": 87, "status": "Applied" } ]
}
```

#### **GET /jobs/{jobId}/skill-gap/{candidateId} — Skill Gap Analysis**

#### ***Response***

```
{
  "candidateId": 201,
  "jobId": 5001,
  "matched": ["Java", "Spring Boot"],
  "partial": ["AWS"],
  "missing": ["Kafka"]
}
```

## 8. Interview Management

### POST /interviews — Schedule Interview

#### Request

```
{
  "applicationId": 9001,
  "interviewType": "Technical",
  "interviewerId": 410,
  "scheduledAt": "2026-10-05T10:00:00+05:30",
  "durationMinutes": 60,
  "mode": "Video"
}
```

#### Response

```
{ "interviewId": 7001, "status": "Scheduled" }
```

#### Error Response

```
{ "errorCode": "INT-001", "message": "Candidate is not in an interview-eligible stage." }
```

### GET /interviews/{interviewId} — Get Interview

#### Response

```
{
  "interviewId": 7001,
  "applicationId": 9001,
  "interviewerId": 410,
  "scheduledAt": "2026-10-05T10:00:00+05:30",
  "status": "Scheduled"
}
```

### PATCH /interviews/{interviewId}/interviewer — Assign Interviewer — added

US-040 is a distinct story from scheduling; separated so an interviewer can be reassigned without rescheduling.

#### Request

```
{ "interviewerId": 415 }
```

#### Response

```
{ "interviewId": 7001, "interviewerId": 415 }
```

### PATCH /interviews/{interviewId}/reschedule — Reschedule Interview — changed from PUT

#### Request

```
{
  "newScheduledAt": "2026-10-07T14:00:00+05:30",
  "reason": "Interviewer conflict"
}
```

#### Response

```
{ "interviewId": 7001, "status": "Rescheduled" }
```

### PATCH /interviews/{interviewId}/cancel — Cancel Interview — changed from DELETE

Modeled as a state change, not a delete — the interview record and history must be retained for audit (BR-006).

#### Request

```
{ "reason": "Position on hold" }
```

#### Response

```
{ "interviewId": 7001, "status": "Cancelled" }
```

### **POST** /interviews/{interviewId}/feedback — Submit Feedback

#### *Request*

```
{
  "technical": 4,
  "communication": 5,
  "problemSolving": 4,
  "domainKnowledge": 3,
  "comments": "Strong Kafka fundamentals.",
  "recommendation": "Hire"
}
```

#### *Response*

```
{ "feedbackId": 8801, "interviewId": 7001, "status": "Submitted" }
```

### **GET** /interviews/{interviewId}/feedback-summary — Consolidated Feedback Summary — added

Ref: FR-INT-06 — aggregates all panelist scorecards for the hiring manager.

#### *Response*

```
{
  "interviewId": 7001,
  "panel": [ { "interviewerId": 410, "recommendation": "Hire", "technical": 4 } ],
  "consolidatedRecommendation": "Hire"
}
```

## 9. Offer Management

### **POST** /offers — Generate Offer

#### *Request*

```
{
  "applicationId": 9001,
  "compensation": { "baseSalary": 1800000, "currency": "INR", "variablePay": 150000 },
  "joiningDate": "2026-11-01"
}
```

#### *Response*

```
{ "offerId": 6001, "status": "Draft" }
```

#### *Error Response*

```
{ "errorCode": "OFF-001", "message": "Offer can only be created for a selected candidate." }
```

### **GET** /offers/{offerId} — Get Offer

#### *Response*

```
{
  "offerId": 6001,
  "applicationId": 9001,
  "status": "Draft",
  "compensation": { "baseSalary": 1800000, "currency": "INR" },
  "joiningDate": "2026-11-01"
}
```

### **PATCH** /offers/{offerId}/approve — Approve Offer

***Request***

```
{ "decision": "Approved", "comments": "Compensation within band." }
```

***Response***

```
{ "offerId": 6001, "status": "Approved" }
```

**PATCH /offers/{offerId}/send — Send Offer**

***Response***

```
{ "offerId": 6001, "status": "Sent", "sentAt": "2026-10-20T09:00:00+05:30" }
```

**PATCH /offers/{offerId}/accept — Accept Offer**

***Response***

```
{ "offerId": 6001, "status": "Accepted", "respondedAt": "2026-10-21T11:00:00+05:30" }
```

**PATCH /offers/{offerId}/decline — Decline Offer**

***Request***

```
{ "declineReason": "Accepted another offer" }
```

***Response***

```
{ "offerId": 6001, "status": "Declined" }
```

*Offer expiration (FR-OFF-05) is a scheduled system job, not a client-invoked endpoint.*

## 10. Background Verification

### **POST /offers/{offerId}/background-check — Initiate Background Check — added**

Missing from the original draft despite being a Must-Have FR module. Ref: FR-BGV-01, FR-BGV-04 (consent required before initiation).

#### *Request*

```
{
  "consentGiven": true,
  "checksRequested": ["Criminal", "Education", "Employment"]
}
```

#### *Response*

```
{ "checkId": 4001, "status": "Initiated" }
```

### **GET /background-checks/{checkId} — Get Background Check Status — added**

Access restricted to authorized roles only (FR-BGV-02).

#### *Response*

```
{ "checkId": 4001, "status": "Completed", "result": "Clear", "discrepancies": [] }
```

## 11. Onboarding & Pre-boarding

### **GET /onboarding/{applicationId}/tasks — Get Onboarding Checklist — added**

Auto-triggered on offer acceptance. Ref: FR-ONB-01, FR-ONB-06.

#### *Response*

```
{
  "applicationId": 9001,
  "tasks": [ { "taskId": 301, "owner": "IT", "description": "Provision laptop & email account", "status": "Pending", "dueDate": "2026-10-30" } ]
}
```

### **PATCH /onboarding/tasks/{taskId} — Update Onboarding Task — added**

#### *Request*

```
{ "status": "Completed" }
```

#### *Response*

```
{ "taskId": 301, "status": "Completed" }
```

*Background Verification and Onboarding & Pre-boarding are scoped pending resolution of the BRD §5.8 vs §7.9 conflict flagged in the Business Domains & Business State Model and Implementation Plan documents.*

## 12. Notifications

### **POST /notifications — Trigger Notification — merged from Send Email + Send SMS**

A single endpoint with a channel field, matching the BRD's "configurable channel" model (FR-21), instead of one endpoint per channel.

#### *Request*

```
{
  "channel": "Email",
  "recipientId": 201,
}
```

```
"template": "APPLICATION_CONFIRMATION",  
"event": "CandidateApplied"  
}
```

**Response**

```
{ "notificationId": 55001, "status": "Sent" }
```

*channel: Email | SMS | InApp. In production this is invoked internally by the workflow engine on configured recruitment events (BR-050), not called directly by end-user clients.*

**GET /notifications?recipientId=201&page=1&size=20 — Notification History**

**Response**

```
{  
  "data": [ { "notificationId": 55001, "channel": "Email", "event": "CandidateApplied",  
"sentAt": "2026-09-01T10:05:00+05:30" } ]  
}
```

## 13. Dashboard & Reporting

### GET /dashboard — Recruitment Dashboard

#### Response

```
{ "openPositions": 25, "applications": 1250, "shortlisted": 180, "interviews": 72, "offers": 18, "hires": 12 }
```

### GET /dashboard/metrics — Hiring Metrics

#### Response

```
{ "timeToHire": 38, "timeToFill": 42, "offerAcceptanceRate": 0.85 }
```

### GET /dashboard/funnel?jobId=5001 — Recruitment Funnel

#### Response

```
{ "applied": 120, "screened": 80, "shortlisted": 30, "interviewed": 15, "offered": 5, "hired": 3 }
```

### GET /dashboard/time-to-hire?department=Engineering&period=Q3-2026 — Time to Hire

#### Response

```
{ "department": "Engineering", "avgTimeToHireDays": 36 }
```

### GET /reports/candidates?format=csv — Candidate Report

#### Response

```
{ "reportId": "RPT-CAND-001", "downloadUrl": "/files/reports/RPT-CAND-001.csv" }
```

*format: json | csv | xlsx (FR-RPT-06). /reports/jobs, /reports/recruiters, /reports/interviews, and /reports/offers follow the identical request/response pattern — omitted here individually to avoid repeating the same shape five times.*

## 14. Administration

### GET /admin/roles — Get Roles

#### Response

```
{ "data": [ { "roleId": 1, "name": "Recruiter" }, { "roleId": 2, "name": "HiringManager" } ] }
```

### POST /admin/roles — Create Role

#### Request

```
{ "name": "InterviewPanelLead", "permissions": ["VIEW_CANDIDATE", "SUBMIT_FEEDBACK"] }
```

#### Response

```
{ "roleId": 8, "message": "Role created." }
```

### POST /admin/users/{id}/roles — Assign Role

#### Request

```
{ "roleId": 2 }
```

#### Response

```
{ "userId": 305, "roles": ["Recruiter", "HiringManager"] }
```

### **PUT /admin/workflow — Configure Workflow**

#### **Request**

```
{ "workflowType": "RequisitionApproval", "stages": ["HiringManager", "Finance", "HRDirector"] }
```

#### **Response**

```
{ "message": "Workflow configuration updated." }
```

### **GET /admin/audit?entityType=Application&entityId=9001 — Audit Logs**

#### **Response**

```
{  
  "data": [ { "user": "Priya Nair", "action": "Shortlisted", "entity": "Application:9001",  
    "timestamp": "2026-09-10T14:22:00+05:30" } ]  
}
```

## 15. External Integrations

### **POST /integrations/outlook/calendar — Outlook Calendar**

Invoked internally by Interview Management on schedule/reschedule, not called directly by end-user clients.

#### **Request**

```
{  
  "interviewId": 7001,  
  "attendees": ["candidate@gmail.com", "interviewer@company.com"],  
  "startTime": "2026-10-05T10:00:00+05:30",  
  "endTime": "2026-10-05T11:00:00+05:30"  
}
```

#### **Response**

```
{ "eventId": "outlook-evt-9981", "status": "Created" }
```

*/integrations/google/calendar, /integrations/teams/meeting, and /integrations/zoom/meeting follow the identical pattern with provider-specific authentication — omitted individually here to avoid repeating the same shape.*

### **POST /integrations/hrms/sync — HRMS Sync**

#### **Request**

```
{ "applicationId": 9001, "trigger": "OfferAccepted" }
```

#### **Response**

```
{ "hrmsEmployeeId": "EMP-88213", "status": "Synced" }
```

## API Security

| Security Requirement | Details                                |
|----------------------|----------------------------------------|
| Authentication       | JWT Bearer Token                       |
| Authorization        | Role-Based Access Control (RBAC)       |
| Encryption           | HTTPS / TLS 1.2+                       |
| Rate Limiting        | 100 requests/minute per user           |
| Input Validation     | Server-side validation on all requests |
| Audit Logging        | Record create/update/delete operations |

## Suggested HTTP Status Codes

| Status Code | Meaning                                |
|-------------|----------------------------------------|
| 200         | Success                                |
| 201         | Resource Created                       |
| 204         | No Content                             |
| 400         | Bad Request                            |
| 401         | Unauthorized                           |
| 403         | Forbidden                              |
| 404         | Resource Not Found                     |
| 409         | Conflict (e.g., duplicate application) |
| 422         | Validation Error                       |
| 500         | Internal Server Error                  |

## API Design Recommendations

- Consistent, resource-oriented endpoints with URI versioning (/v1).
- Filtering, sorting, and pagination on all collection endpoints (GET /jobs, GET /applications).
- Standardized error response shape across all endpoints (errorCode + message).
- Idempotent methods used where appropriate (PUT for full replace, PATCH for partial/state-transition updates).
- Full specification maintained as an OpenAPI (Swagger) document alongside this reference.

## Appendix — Document Ownership & API Lifecycle

Request/response contracts are not finalized during requirements gathering. The Business Analyst defines business requirements, functional requirements, data elements, validation rules, business rules, and acceptance criteria (already captured in the BRD/FRD and Business Rules & Acceptance Criteria documents); the Solution Architect or Technical Lead then translates those into the endpoints, payloads, and error handling in this document. In Agile delivery, the API contract for a story is typically added during backlog refinement/sprint planning, once the story's acceptance criteria are agreed.

| Phase                             | BA Responsibility                    | Request/Response Defined?                      |
|-----------------------------------|--------------------------------------|------------------------------------------------|
| Business Requirements (BRD)       | Capture business needs and processes | No                                             |
| Functional Requirements (FRD/SRS) | Define what the system should do     | High-level only, sometimes sample payloads     |
| API Design / Solution Design      | Define technical contracts           | Yes — request, response, headers, status codes |
| Development                       | Implement the API                    | Uses the specification                         |
| Testing                           | Validate the API                     | Tests against the defined contract             |